

CMP-142 Object Oriented Programming

Lab-11

Topics:

- Recursion
- Revision

Instructions!

- This is a **graded lab**, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed to ask how he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
- Indent your code properly.
- Use meaningful variable and function names. **Use the camelCase notation.**
- Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
- **Do NOT use any global or static variable(s).** However, global named constants may be used.
- Make sure that there are **NO dangling pointers or memory leaks** in your programs.
- ***Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.***

[Task 1] Economy Simulation

[50 Marks]

You are tasked with designing a **Dynamic Economy Simulation** system. The system involves various **Companies**, **Banks**, **Employees**, and **Customers**, each interacting with one another within a simulated economy. The system should model financial transactions, the creation and distribution of wealth, and the effects of economic events (e.g., inflation, bank runs, layoffs, etc.).

Entities to Implement:

1. **Person** (Abstract Base Class):
 - Attributes:
 - **name** (string): The name of the person.
 - **age** (int): The age of the person.
 - **balance** (float): The current balance of the person's bank account.
 - **employment_status** (bool): Whether the person is employed or not.

- Methods:
 - `deposit(float amount)` - Adds money to the person's balance.
 - `withdraw(float amount)` - Subtracts money from the person's balance.
 - `isEmployed()` - Returns whether the person is employed or not.
 - `setEmploymentStatus(bool status)` - Sets the employment status of the person.

2. **Employee** (Derived Class from **Person**):

- Attributes:
 - `salary` (float): The salary of the employee.
 - `company` (Company*): The company that employs the person.
- Methods:
 - `receiveSalary()` - Deposits the employee's salary into their bank account (if employed).
 - `work()` - Performs the employee's work at the company, generating value.

3. **Customer** (Derived Class from **Person**):

- Attributes:
 - `spendingPower` (float): How much money the customer is able to spend.
- Methods:
 - `makePurchase(float amount)` - Customer makes a purchase, deducting money from their balance.

4. **Company** (Class):

- Attributes:
 - `name` (string): The name of the company.
 - `revenue` (float): The total revenue of the company.
 - `employees` (vector<Employee*>): List of employees working at the company.
 - `profitMargin` (float): The percentage of revenue retained after expenses.
- Methods:
 - `hireEmployee(Employee &employee)` - Adds an employee to the company's workforce.
 - `fireEmployee(Employee &employee)` - Removes an employee from the company.
 - `payEmployees()` - Pays salaries to all employees.
 - `generateRevenue()` - Simulates the company earning revenue.

- `distributeProfit()` - Distributes profit to the company's employees based on their contribution.

5. **Bank** (Class):

- Attributes:

- `name` (string): The name of the bank.
- `accounts` (`Person*` `accounts`): A list of people who have accounts in the bank.
- `interestRate` (float): The interest rate on savings accounts.

- Methods:

- `openAccount(Person &person)` - Opens an account for a person.
- `closeAccount(Person &person)` - Closes a person's bank account.
- `applyInterest()` - Applies interest to all accounts based on the interest rate.

6. **EconomicEvent** (Class):

- Attributes:

- `name` (string): The name of the event (e.g., "Recession", "Inflation", "Boom").
- `effect` (string): Description of the economic effect.
- `impactFactor` (float): How strongly the event will impact the economy (e.g., reducing income, affecting company revenue).

- Methods:

- `triggerEvent()` - Triggers the economic event, affecting companies, employees, and customers accordingly.

7. **Economy** (Class):

- Attributes:

- `companies` (`Company*` `companies`): List of companies operating in the economy.
- `people` (`Customer*` `customers`): List of people in the economy (employees and customers).
- `banks` (`Bank*` `banks`): List of banks in the economy.
- `events` (`EconomicEvent*` `events`): A list of events happening in the economy.

- Methods:

- `addCompany(Company &company)` - Adds a company to the economy.
 - `addPerson(Person &person)` - Adds a person (either customer or employee) to the economy.
 - `addBank(Bank &bank)` - Adds a bank to the economy.
 - `simulateEconomy()` - Runs a simulation of the economy where companies generate revenue, employees are paid, customers make purchases, and economic events can trigger.
 - `triggerEconomicEvent(EconomicEvent &event)` - Triggers a specific economic event, altering the dynamics of the economy.
 - `runSimulationRound()` - Executes a round of actions such as hiring/firing, revenue generation, salary payment, purchases, and event effects.
-

Detailed Class Behavior:

1. Employee's Role:

- **Work:** Employees generate value for their companies by working. The company's revenue increases as a result.
- **Receive Salary:** Employees get paid based on their salary and the company's profitability. Salaries are deposited directly into their bank accounts.

2. Company's Role:

- **Hire and Fire Employees:** Companies can hire or fire employees based on their needs and economic conditions.
- **Revenue Generation:** Companies generate revenue based on their operations and the work done by employees.
- **Profit Distribution:** After paying expenses, the company distributes its profits among its employees according to their contributions.

3. Bank's Role:

- **Accounts Management:** Banks manage the bank accounts of employees, customers, and others.
- **Interest Application:** Banks apply interest to the accounts, increasing the balances of the account holders over time.

4. EconomicEvent's Role:

- **Impact on Economy:** Events like "Inflation," "Boom," or "Recession" can significantly alter how people and companies behave. For example, in a

recession, employees may receive lower salaries, companies may generate less revenue, and customers may have less spending power.

5. **Economy's Role:**

- **Simulating Interactions:** The **Economy** class orchestrates the simulation. It manages companies, people, and banks, ensuring that all elements interact with each other.

Expected Output:

John has been hired by TechCorp
Jane has been hired by RetailMart
John opened an account at CentralBank
Jane opened an account at CentralBank
TechCorp generated revenue of \$1000
TechCorp is distributing \$200 as profit.
John receives profit.
Jane receives profit.
RetailMart generated revenue of \$1000
RetailMart is distributing \$300 as profit.
John receives profit.
Jane receives profit.
John received interest: \$50
Jane received interest: \$40
John made a purchase of \$100
Jane made a purchase of \$100
Economic Event: Recession is triggered. Effect: Economic downturn, reduced spending
TechCorp revenue adjusted due to economic impact.
RetailMart revenue adjusted due to economic impact.
John spending power adjusted.
Jane spending power adjusted.
TechCorp generated revenue of \$700
TechCorp is distributing \$140 as profit.
John receives profit.
Jane receives profit.
RetailMart generated revenue of \$700
RetailMart is distributing \$210 as profit.
John receives profit.
Jane receives profit.

Explanation of the Output:

1. **Hiring and Account Creation:** The system starts by hiring employees (John and Jane) and opening accounts for them at the bank (CentralBank).
2. **Revenue Generation & Profit Distribution:** Each company generates revenue based on the number of employees, and they distribute profits to employees.
3. **Interest Application:** The bank applies interest to the accounts of John and Jane.
4. **Purchases:** John and Jane make purchases of \$100 each.
5. **Economic Event:** A **Recession** event is triggered, reducing the revenue of companies and spending power of customers. This is reflected in the subsequent rounds.

[Task 2] Calculate Maximum Product of Two Elements in an Array

Problem Statement: You are given an array of integers. Write a recursive function to find the maximum product of two distinct elements in the array.

For example, given an array of integers:

[10, -10, 3, -1, 2, 7]

The maximum product is $10 * 7 = 70$.

Your function should return the maximum product of any two distinct elements in the array. And remember that you are not allowed to sort the array. Your solution should be recursive.

Function Signature:

```
int maxProduct(int arr[], int n);
```

Where:

- `arr[]` is the input array.
- `n` is the size of the array.

Example 1:

Input:

```
arr = [10, -10, 3, -1, 2, 7]
n = 6
```


Output:

70

Example 2:

Input:

```
arr = [1, 3, 5, 2, 8]  
n = 5
```

Output:

40